

Memoria Proyecto Final

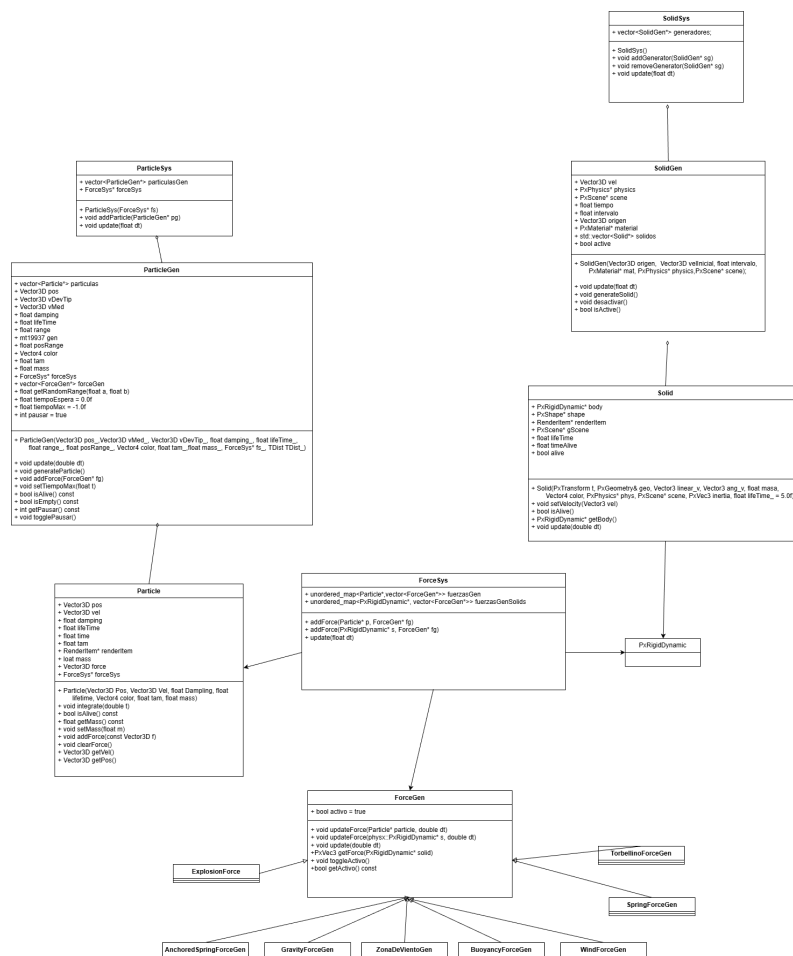
Iván Palomino Rodriguez

● Objetivo del juego

El juego propuesto se trata de un lanzamiento a canasta. El jugador deberá encestar en la canasta para así anotar puntos. Cada cierto tiempo el juego cambiará aplicando un efecto aleatorio en el como: pelotas más grandes o pequeñas, la canasta se moverá de lado a lado, la canasta subirá y bajará como un muelle, habrá una zona de viento o habrá objetos que te impedirán encestar. El objetivo es llegar a 10 puntos en menos de 60 segundos, el jugador podrá seguir anotando hasta que pasen los 60.

● Diagrama de clases

Como se ve tan pequeño, dejo la imagen en el github, también dejo un archivo .drawio que desde la página web de navegador [Drawio](#) se puede abrir el archivo.



● Ecuaciones físicas usadas

-El lanzamiento de la pelota se realiza mediante una velocidad inicial proporcional a la fuerza acumulada por el jugador, simulando un tiro parabólico.

$$v_0 = d * F$$

Con un pequeño ajuste vertical:

$$v_{0y} = v_{0y} + 0.2 * F$$

-El tensor de inercia de las pelotas se calcula usando la expresión analítica de una esfera maciza

$$I = \frac{2}{5}mr^2$$

-La explosión usa fuerza inversamente proporcional al cuadrado de la distancia y con decaimiento exponencial en el tiempo.

$$F = K/r^2 * e^{-t/\tau} * r$$

-El viento se implementa como una fuerza dependiente de la diferencia de velocidades entre el sólido y el viento, combinando términos lineales y cuadráticos.

$$F = k_1(v_{\text{viento}} - v) + k_2(v_{\text{viento}} - v) |v_{\text{viento}} - v|$$

-Se aplica una fuerza constante horizontal para simular un desplazamiento lateral de la canasta

$$F = (F_x, 0, 0)$$

-El efecto del muelle se basa en la ley de Hooke, aplicando una fuerza proporcional a la elongación del muelle respecto a su longitud de reposo

$$F = -k(x - x_0)d$$

● Efectos

-La canasta se mueve lateralmente aplicando una fuerza constante horizontal.

-Efecto de viento que introduce una zona de viento que aplica fuerzas a las pelotas, modificando su trayectoria durante el vuelo.

-Efecto de cambiar tamaño, cambia de masa tamaño y color a la pelota, se hace más grande o más pequeña

-Efecto de muelle que provoca que las canastas vayan de arriba y abajo con efecto de muelle.

-Efecto de objetos voladores, durante este efecto se generan sólidos rígidos en movimiento que actúan como obstáculos dinámicos dentro del escenario.

- **Requisitos/Efectos/Ecuaciones en el código**

-Requisito: Dos generadores de partículas distintos) →

Implementado en:

“main.cpp” líneas 112 y 113 con distinta distribución y en “TiroCanasta.cpp” línea 19 y 133

Explicación: los dos primeros sistemas de partículas saltan cuando hay una victoria en el juego simulando confeti, en la línea 19 el generador de partículas simula donde está la zona del viento y el generador de partículas de la línea 133 simula una explosión de pelota cuando el jugador encuesta.

-Requisito: Gestión de creación y destrucción de instancias) →

Implementado en :

“ParticleGen.cpp” líneas 62-63, creación de partículas en el vector, línea 39,

Comprobación de espacio y tiempo y destrucción de estas.

Para los generadores de sólidos ocurre lo mismo, “SolidGen.cpp” líneas 45-46 y línea comprobación de destrucción línea 27.

Para las pelotas creación en “TiroCanasta.cpp” líneas 221-222 y destrucción en línea 155 y

61 Para las canastas creación en “CanastaManager” en método addBasket y destrucción en método removeBasket o clearB.

-Requisito: Destrucción de todos los elementos al salir de la escena) →

Implementación: “Main.cpp” método cleanupPhysics y las diferentes destructoras de

TiroCanasta.cpp, CanastaManager.cpp, ForceSys.cpp, ParticleSys.cpp, SolidSys.cpp y

GameEffectManager.cpp.

-Requisito: Partículas con distinta masa) →

Implementación en:

“Particle.cpp” en la constructora, los diferentes generadores de partículas tienen distintas masas.

-Requisito: Un sistema de sólidos rígidos) →

Implementación en:

Clase “SolidSys” se encarga de gestionar los generadores de sólidos, actualizando cada conjunto el

cual previamente se tiene que haber añadido al sistema con el método void addGenerator(SolidGen* sg); . También el sistema borra los generadores que están controlados por tiempo.

-Requisito: Sólidos rígidos con distinto tamaño, forma, masa y tensor de inercia) →

Implementación en:

Clase “TiroCanasta.cpp” línea 185-226 método void crearBola(Vector3D pos, Vector3D dir, float fuerza); creo tres posibles tipos de sólido distintos con masa, tamaño, color y tensor de inercia distintos.

Para ver el cambio de forma tengo un generador de sólido que genera sólidos cuadrados en “EfectoSólidosVoladores.cpp”.

-Requisito: Dos generadores de fuerzas diferentes) →

Implementación en:

Clase "TiroCanasta.cpp" creo una fuerza de viento en línea 14 y otra fuerza de explosion en la línea 30. En las respectivas clases de "ZonaVientoGen.cpp" y "ExplosionForce.cpp" se puede como se crean las diferentes fuerzas

-Requisito: Interacción con el usuario a través del teclado y/o ratón) →

Implementación en:

Clase "main.cpp" línea 284 método keyPress y línea 318 método keyRelease. Al mantener el espacio se cargará el disparo, y si sueltas el espacio dispara una pelota, con la tecla 'e' se a avanzará entre estados de menús.

-Requisito: Un ejemplo de muelle o de flotación) →

Implementación en:

Clase "EfectoMuelle" línea 44, en la línea 43 se declara la fuerza del muelle que está definida en la clase "SpringForce.cpp".

-Efecto: La canasta se mueve lateralmente) →

Implementación en:

Clase "EffectBasketLado.cpp"

-Efecto: Zona de viento) →

Implementación en:

Clase "TiroCanasta.cpp" línea 14, se activa y desactiva en la clase "EffectWind.cpp"

-Efecto: Cambiar de tamaño) →

Implementación en:

Clase "TiroCanasta.cpp" línea 121, metodo void cambiarBola(int cb) , se activa y desactiva en la clase "EfectoBolaGrande.cpp" y "EfectoBolaPequeña.cpp"

-Efecto: Muelle canasta va de arriba a abajo) →

Implementación en:

Clase "EfectoMuelle.cpp"

-Efecto: Objetos voladores) →

Implementación en:

Clase "EffectSolidosVoladores.cpp"

-Ecuaciones Físicas: Lanzamiento de pelotas) →

Implementación en:

Clase "TiroCanasta.cpp", líneas 208-216.

-Ecuaciones Físicas: Explosion) →

Implementación en:

Clase "ExplosionForce.cpp", método void updateDorce(Particle* p, double dt).

-Ecuaciones Físicas: Viento) →

Implementación en:

Clase "ZonaDeVientoGen.cpp", método void updateDorce(Particle* p, double dt).

-Ecuaciones Físicas: Movimiento lateral) →

Implementación en:

Clase "EffectbasketLado.cpp", metodo void update(float t).

-Ecuaciones Físicas: Muelle) →

Implementación en:

Clase "SpringForce.cpp", método void updateDorce(Particle* p, double dt).

● Manual

Manteniendo el espacio -- Cargar disparo (Cuanto más pulses el espacio más fuerza sale la pelota)

Soltar espacio -- Disparar

Apretando click izquierdo y moviendo ratón -- Movimiento de cámara para poder apuntar

Pulsando la tecla 'e' se avanzará entre menús

● Efectos Extra

-Sistema de estados, hay un menú principal, la pantalla de juego y las pantallas de derrota y victoria.

-Sistema de Canastas que controlan las canastas.

-Sistema de tiro de canastas que controlan el tiro.

-Sistema de efectos que van cambiando cada cierto tiempo durante el juego.

-Detección si entra la pelota en la canasta.

-Paneles de texto indicando puntos, tiempo restante y estado actual.

-Explosion, para indicar que se encestó una pelota se genera un sistema de partículas el cual es afectado por una fuerza de explosion.